

Real-Time Computer Systems and Computer Architecture

Laboratory 1

Registers, GPIO, Timers and Interrupts

Student: Student Name

Glasgow ID: Glasgow ID

UESTC ID: UESTC ID

September 4, 2026

Abstract

This laboratory investigates GPIO register manipulation and interrupt-driven timing in an STM32 embedded system. Individual register fields are configured through masking and shifting, a GPIO input expression is evaluated at bit level, and direct IDR and ODR operations are used to associate a push-button input with an LED output. A timer-driven application then combines an external button interrupt with periodic TIM2 update interrupts to start and stop LED blinking at 1.5 s intervals. The results demonstrate the progression from elementary bitwise operations to event-driven peripheral control.

1 Introduction

Memory-mapped registers provide direct control of microcontroller peripherals. GPIO configuration fields determine pin modes, whereas input and output data registers expose pin states to software. Masking and shifting allow one field to be accessed without modifying unrelated register bits.

Timers and interrupts extend this register-level control to time-dependent behavior. An external interrupt records the button event, and an internal timer interrupt performs the periodic LED update without blocking program execution. The following tasks examine these mechanisms through register configuration, input evaluation, direct LED control, and an interrupt-driven blinking sequence.

2 Task 1: GPIOB Pin 5 Register Configuration

The task uses the four-bit GPIOx_CRL pin field. Pin 5 occupies bits 23:20. General-purpose push-pull output mode requires CNF[1:0] = 00, and the 50 MHz output selection requires MODE[1:0] = 11; therefore the complete nibble is 0011.

2.1 Step-by-step bitwise operations

Let the untouched register bits be written as x. The field mask is

$$M = 0xF \ll 20 = 0x00F00000.$$

The starting register is represented symbolically as

xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx.

First, clear bits 23:20:

```
1 GPIOB->CRL &= ~0x00F00000U;
```

The result is

xxxxxxxx 0000xxxx xxxxxxxx xxxxxxxx.

Second, place 0011 in the field:

```
1 GPIOB->CRL |= 0x00300000U;
```

The result is

xxxxxxxx 0011xxxx xxxxxxxx xxxxxxxx.

Figure 1 shows that only the PB5 field changes.

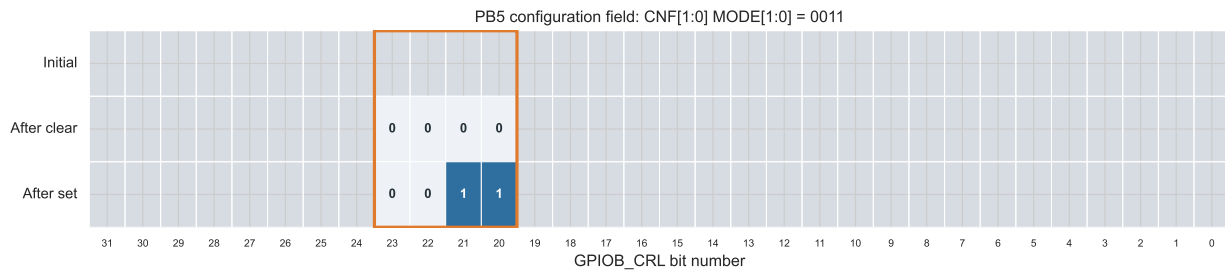


Figure 1: Bit-field progression for GPIOB pin 5. Grey cells are preserved bits.

2.2 Minimum-step code

The clear and set transformations can be combined into one read-modify-write statement:

```
1 GPIOB->CRL = (GPIOB->CRL & ~(0xFU << 20U)) | (0x3U << 20U);
```

The mask preserves all fields other than PB5, while the encoded value writes 0011 into bits 23:20.

3 Task 2: Evaluating a GPIO Input Bit

The expression is

```
1 bool Pinx = (GPIOx->IDR & (1U << GPIOx_PIN_N)) != 0U;
```

It evaluates in three operations:

1. $1U \ll GPIOx_PIN_N$ creates a mask with a single one at the selected pin position.
2. Bitwise AND removes every other input bit and retains only the selected pin state.
3. Comparison with zero converts the masked value to false or true.

For pin 13, the mask is $1 \ll 13 = 0x2000$. If $GPIOC->IDR$ is $0x2400$, then

$$0x2400 \& 0x2000 = 0x2000,$$

which is non-zero, so `Pinx` is true. If the register value is $0x0400$, the AND result is zero and `Pinx` is false.

4 Task 3: Button-Controlled LED with IDR and ODR

B1 is read from bit 13 of $GPIOC->IDR$. A press produces the active input level used by the falling-edge configuration. PA5 is then set or cleared in $GPIOA->ODR$; the other output bits remain unchanged.

Listing 1: Task 3 register-level implementation.

```
1 #include "task3_polling.h"
2
3 #include <stdbool.h>
4
5 #include "main.h"
6
7 void Task3_UpdateLedFromButton(void)
8 {
9     bool button_pressed = (GPIOC->IDR & B1_Pin) == 0U;
10
11     if (button_pressed)
```

```

12 {
13     GPIOA->ODR |= LD2_Pin;
14 }
15 else
16 {
17     GPIOA->ODR &= ~LD2_Pin;
18 }
19 }

```

The Boolean expression normalizes the button state, after which the two ODR operations directly implement the required LED-on and LED-off behavior.

5 Task 4: Interrupt-Driven 1.5 s Blinking

5.1 Timer calculation

TIM2 receives a 32 MHz clock because APB1 uses a divider of one. A 1 kHz counter tick is formed with a division factor of 32,000:

$$f_{\text{counter}} = \frac{32\,000\,000}{31999 + 1} = 1000 \text{ Hz.}$$

Counting 1,500 ticks produces the requested interval:

$$T_{\text{update}} = \frac{1499 + 1}{1000} = 1.5 \text{ s.}$$

TIM2 values

Timer	TIM2
Prescaler register	31999
Counter period register	1499
Counter mode	Up
Update interrupt	Enabled

The generated initialization applies these values:

```

1 htim2.Instance = TIM2;
2 htim2.Init.Prescaler = 31999;
3 htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
4 htim2.Init.Period = 1499;
5 htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
6 htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
7 HAL_TIM_Base_Init(&htim2);

```

5.2 External and internal interrupt behavior

The PC13 falling edge calls HAL_GPIO_EXTI_Callback. The first accepted press turns LD2 on, resets TIM2, and starts update interrupts. Each TIM2 period callback toggles PA5. The next accepted press stops TIM2 with HAL_TIM_Base_Stop_IT and clears PA5. A 30 ms time gate suppresses the extra edges produced by the mechanical button.

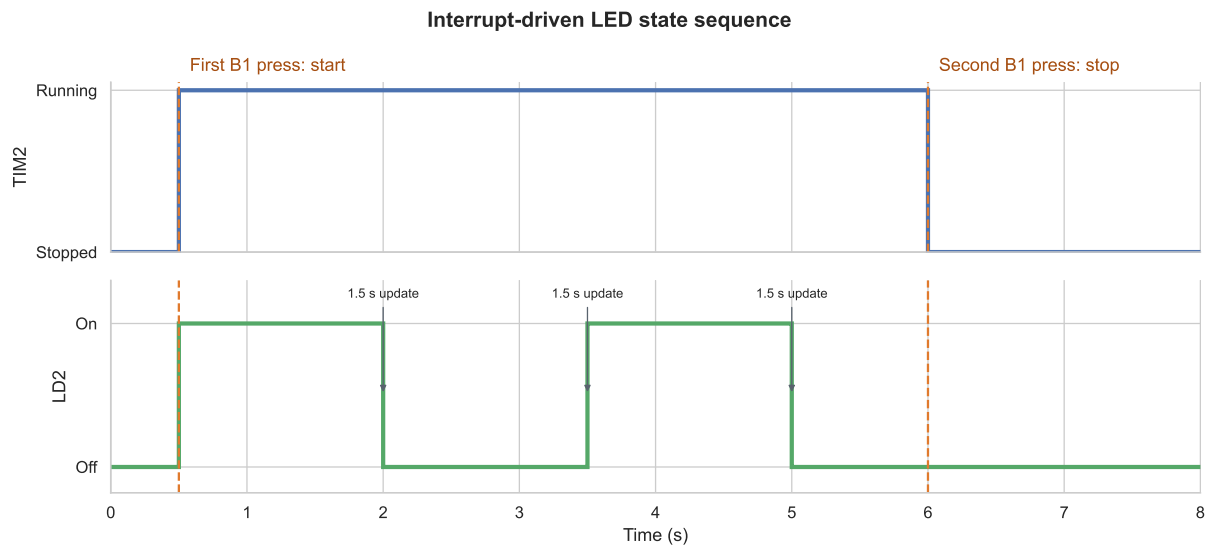


Figure 2: Example Task 4 state sequence. Timer updates toggle LD2 every 1.5 s.

Listing 2: Task 4 application module.

```

1  #include "lab1_app.h"
2
3  #include <stdbool.h>
4
5  #include "main.h"
6
7  #define BUTTON_DEBOUNCE_MS 30U
8
9  static TIM_HandleTypeDef *timer;
10 static bool blinking;
11 static uint32_t last_button_tick;
12
13 static void StartBlinking(void)
14 {
15     blinking = true;
16     GPIOA->ODR |= LD2_Pin;
17     __HAL_TIM_SET_COUNTER(timer, 0U);
18     HAL_TIM_Base_Start_IT(timer);
19 }
20
21 static void StopBlinking(void)
22 {
23     blinking = false;
24     HAL_TIM_Base_Stop_IT(timer);
25     GPIOA->ODR &= ~LD2_Pin;
26 }
27
28 void Lab1_AppInit(TIM_HandleTypeDef *blink_timer)
29 {
30     timer = blink_timer;
31     blinking = false;
32     last_button_tick = HAL_GetTick() - BUTTON_DEBOUNCE_MS;
33     GPIOA->ODR &= ~LD2_Pin;
34 }
35
36 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
37 {

```

```
38  uint32_t now;
39
40  if (GPIO_Pin != B1_Pin)
41  {
42      return;
43  }
44
45  now = HAL_GetTick();
46  if ((now - last_button_tick) < BUTTON_DEBOUNCE_MS)
47  {
48      return;
49  }
50  last_button_tick = now;
51
52  if (blinking)
53  {
54      StopBlinking();
55  }
56  else
57  {
58      StartBlinking();
59  }
60 }
61
62 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
63 {
64     if ((htim->Instance == TIM2) && blinking)
65     {
66         GPIOA->ODR ^= LD2_Pin;
67     }
68 }
```

The main loop executes `__WFI()` and contains no delay call. All LED timing is generated by the timer update interrupt, while the button transition is captured by EXTI.

6 Conclusion

The laboratory established a direct relationship between register-level operations and interrupt-driven embedded behavior. Masking and shifting configured an individual GPIO field while preserving neighbouring fields, and the IDR expression reduced a selected input bit to a Boolean state. Direct ODR manipulation then produced the required button-controlled LED response. Finally, the combination of a button external interrupt and TIM2 update interrupts implemented a non-blocking 1.5 s blink sequence that starts and stops on successive button presses. These results demonstrate how bitwise register access, peripheral timers, and interrupt callbacks can be integrated into a concise event-driven application.